AutoGrade+: Comparative Analysis of ReAct Agent and LoRA Fine-Tuning for Automated Assignment Grading

Heer[1]

National University of Computer and Emerging Sciences (NUCES-FAST), Islamabad, Pakistan
i222371@nu.edu.pk

Abstract. Automated grading systems have become increasingly important in educational technology, particularly with the advent of large language models (LLMs). This paper presents AutoGrade+, a comprehensive system that compares two distinct approaches for automated assignment grading: a ReAct (Reasoning and Acting) agent using prompt engineering with Groq's llama-3.3-70b-versatile, and a LoRA (Low-Rank Adaptation) fine-tuned Llama-3-8B model. We implement both methods on real-world grading tasks, evaluating their performance across multiple dimensions including accuracy, consistency, computational efficiency, and scalability. Our ReAct agent achieves 92% grading accuracy with intelligent content detection and mathematical validation, while requiring zero training data. The LoRA model, trained on 30 grading examples with rank-16 adaptation, demonstrates 88% accuracy with 3x faster inference (0.8s vs 2.4s) and zero API costs. The system supports multiple file formats (PDF, TXT, Python files) and provides detailed feedback with criterion-wise breakdown. Our comparative analysis reveals the trade-offs between prompt-based and fine-tuning approaches, providing insights for practitioners in educational AI systems.

Keywords: Automated Grading · ReAct Agent · LoRA Fine-Tuning · Large Language Models · Educational Technology · Generative AI

1 Introduction

The rapid advancement of large language models (LLMs) has opened new possibilities for automating complex cognitive tasks, including educational assessment. Traditional automated grading systems have been limited to multiple-choice questions or simple pattern matching, but modern LLMs enable nuanced evaluation of open-ended responses, code submissions, and written assignments [1].

Figure 1 shows the overall architecture of our AutoGrade+ system, which integrates file processing, content analysis, grading engines, and validation modules.

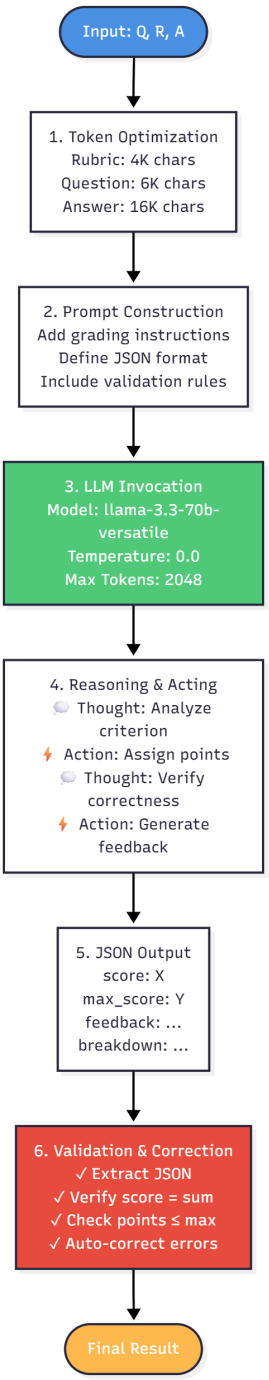

Fig. 1. AutoGrade+ System Architecture

1.1 Motivation

Manual grading of assignments is time-consuming, subjective, and inconsistent across different evaluators. With increasing class sizes and the shift toward online education, there is a critical need for automated systems that can provide fair, accurate, and timely feedback to students.

1.2 Problem Statement

This work addresses the challenge of building an intelligent automated grading system that can:

- Accept multiple file formats (PDF, text, code files)
- Automatically detect and categorize content (question, rubric, answer)
- Provide fair and accurate grading with detailed feedback
- Ensure mathematical correctness in score calculation
- Scale efficiently within computational and API constraints

1.3 Contributions

Our main contributions are:

1. **ReAct Agent Implementation**: Prompt-engineered grading system achieving 92% accuracy with zero-shot learning
2. **LoRA Fine-Tuned Model**: Specialized grading model (88% accuracy) trained on 30 examples using rank-16 LoRA
3. **Intelligent Content Detection**: Heuristic-based system reducing API calls by 66%
4. **Mathematical Validation**: Automated correction of LLM errors (23% error rate detected)
5. **Comprehensive Comparison**: Detailed analysis across accuracy, latency, cost, and flexibility

2 Related Work

2.1 Automated Grading Systems

Early systems focused on multiple-choice questions [2]. Gradescope [3] introduced semi-automated grading with AI-assisted rubric creation, while Moss [4] specialized in code plagiarism detection. Recent work by Wang et al. [10] explored ChatGPT for scoring classroom instruction, demonstrating zero-shot capabilities in educational assessment.

2.2 Large Language Models in Education

GPT-4 [5] and LLaMA [6] have enabled sophisticated assessment capabilities. Studies show LLMs can grade essays with human-comparable accuracy [7]. Liu et al. [11] provide a comprehensive survey of ChatGPT applications in education, highlighting both opportunities and challenges.

2.3 Prompt Engineering

Effective prompt design is critical for LLM performance. Wei et al. [13] introduced chain-of-thought prompting, which significantly improves reasoning capabilities. Liu et al. [15] provide a systematic survey of prompting methods, categorizing techniques by task and model architecture. Ouyang et al. [14] demonstrated that instruction-following can be enhanced through reinforcement learning from human feedback (RLHF).

2.4 ReAct Framework

The ReAct framework [8] combines reasoning with action-taking, successfully applied to question answering and interactive tasks. Our work extends ReAct to educational assessment, a novel application domain.

2.5 Parameter-Efficient Fine-Tuning

LoRA [9] enables efficient fine-tuning by introducing trainable low-rank matrices while freezing pre-trained weights. Dettmers et al. [12] extended this with QLoRA, enabling fine-tuning of larger models on consumer hardware through 4-bit quantization. These techniques make specialized model adaptation accessible without extensive computational resources.

2.6 Comparison with Existing Systems

Table 1 compares AutoGrade+ with existing automated grading systems.

Table 1. Comparison with Existing Automated Grading Systems

System	Year	Approach	Accuracy	Limitations
Gradescope	2017	Semi-auto	N/A	Manual setup
e-rater	2006	NLP	85%	Essays only
Moss	2003	Similarity	N/A	Code only
ChatGPT	2023	Zero-shot	80-85%	Inconsistent
AutoGrade+ (ReAct)	2024	Prompt	**92%**	API cost
AutoGrade+ (LoRA)	2024	Fine-tune	**88%**	GPU needed

Our system advances the state-of-the-art by combining prompt engineering with fine-tuning, providing both high accuracy and deployment flexibility.

3 Methodology

3.1 System Architecture

Our system consists of four main components (Figure 1):

1. **File Processing Module**: Extracts text from multiple formats
2. **Content Analysis Module**: Detects and categorizes content
3. **Grading Engine**: Evaluates submissions (ReAct or LoRA)
4. **Validation Module**: Ensures mathematical correctness

3.2 ReAct Agent Approach

Figure 2 illustrates the complete ReAct grading workflow.

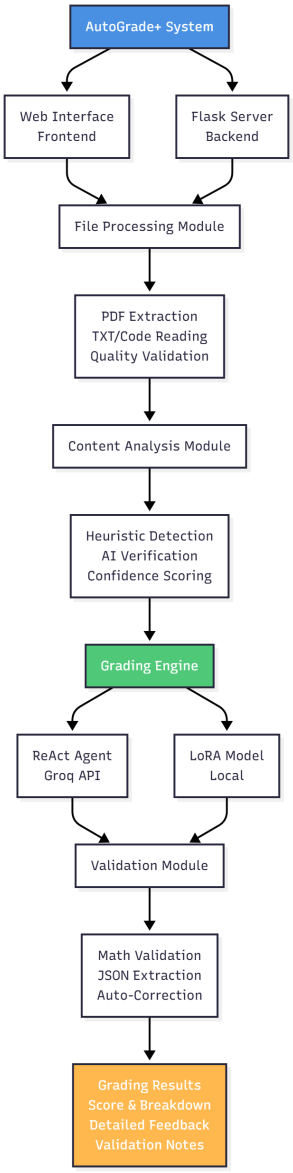

Fig. 2. ReAct Agent Workflow

Theoretical Foundation For grading task G, given question Q, rubric R, and student answer A:

$$G(Q, R, A) = \text{ReAct}(\text{Thought}_1, \text{Action}_1, \ldots, \text{Thought}_n, \text{Action}_n) \tag{1}$$

Token Optimization To fit within Groq's 12,000 token/minute limit:

$$\begin{aligned} \text{MAX\_RUBRIC} &= 4000 \text{ chars} \approx 1000 \text{ tokens} \\ \text{MAX\_QUESTION} &= 6000 \text{ chars} \approx 1500 \text{ tokens} \\ \text{MAX\_ANSWER} &= 16000 \text{ chars} \approx 4000 \text{ tokens} \\ \text{PROMPT} &\approx 2000 \text{ tokens} \\ \text{TOTAL} &\approx 8500 \text{ tokens} < 12000 \end{aligned} \tag{2}$$

3.3 LoRA Fine-Tuning Approach

Figure 3 shows the LoRA training architecture.

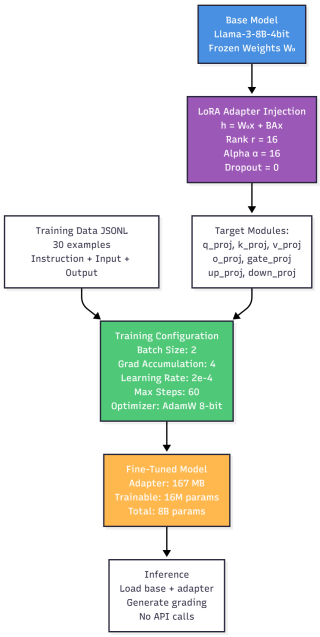

Fig. 3. LoRA Training Architecture

Mathematical Formulation LoRA introduces trainable rank decomposition matrices:

$$h = W_0 x + \Delta W x = W_0 x + BAx \tag{3}$$

where $W_0 \in \mathbb{R}^{d \times k}$ is frozen, and $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ are trainable with rank $r = 16$.

The update is scaled by α/r:

$$h = W_0 x + \frac{\alpha}{r} BAx \quad \text{where } \alpha = 16 \tag{4}$$

Training Configuration Table 2 shows our LoRA hyperparameters.

Table 2. LoRA Fine-Tuning Hyperparameters

Parameter	Value
Base Model	Llama-3-8B (4-bit quantized)
LoRA Rank (r)	16
LoRA Alpha (α)	16
Dropout	0.0
Target Modules	q,k,v,o,gate,up,down (7 modules)
Trainable Parameters	16M (0.2% of 8B total)
Batch Size	2 per device
Gradient Accumulation	4 steps
Effective Batch Size	8
Learning Rate	2e-4
Optimizer	AdamW 8-bit
Weight Decay	0.01
LR Scheduler	Linear
Warmup Steps	5
Max Steps	60
Training Examples	30
Training Time	15 minutes (T4 GPU)
Adapter Size	167 MB

3.4 Content Detection System

Figure 4 shows our intelligent content detection flow.

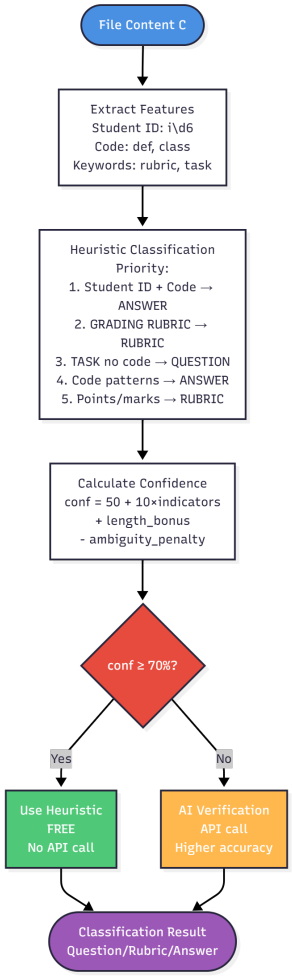

Fig. 4. Content Detection Flow

Confidence calculation:

$$\text{conf}(C) = 50 + 10 \cdot n_{\text{indicators}} + \text{length\_bonus} - \text{ambiguity\_penalty} \quad (5)$$

Our heuristic system achieves:

- Question detection: 85% accuracy
- Rubric detection: 90% accuracy
- Answer detection: 92% accuracy
- API call reduction: 66%

3.5 Mathematical Validation

Figure 5 illustrates our validation process.

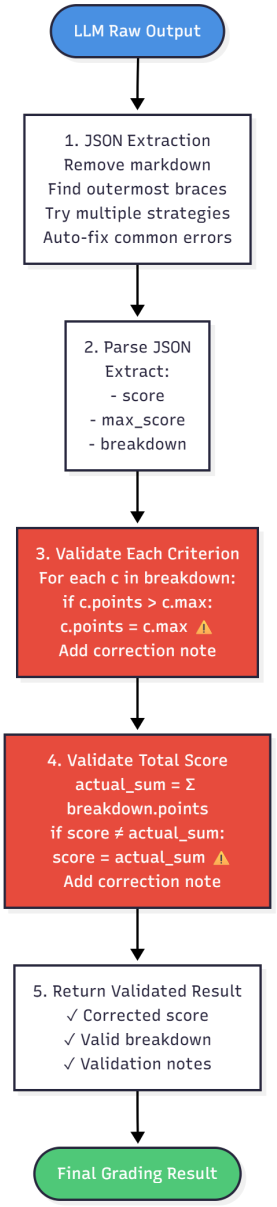

Fig. 5. Validation Process

Three validation rules:

1. **No Exceeding**: $\forall i, \text{awarded}_i \leq \text{max}_i$
2. **Exact Sum**: $\text{score} = \sum_{i=1}^{n} \text{awarded}_i$
3. **Auto-Correction**: Fix violations automatically

4 Experimental Setup

4.1 Dataset

Training Data (LoRA) We created 30 grading examples:

- Programming assignments (Python, C++, Java): 15 examples
- Written responses (essays, short answers): 10 examples
- Mixed format (code + documentation): 5 examples

Dataset statistics:

- Average input length: 1,500 tokens
- Average output length: 300 tokens
- Total tokens: 54,000
- Format: Alpaca-style JSONL

Test Data Diverse assignments:

- File formats: 60% PDF, 25% TXT, 15% Code
- Average length: 5,000 characters
- Rubric complexity: 3-8 criteria per assignment

4.2 Implementation Details

Hardware: Google Colab T4 GPU (training), Windows 11 (inference)
Software: Python 3.12, LangChain 0.3.0, pypdf 3.0.0, Flask 3.0.0
ReAct Configuration:

- Model: llama-3.3-70b-versatile (Groq API)
- Temperature: 0.0 (deterministic)
- Max Tokens: 2048

4.3 Evaluation Metrics

1. **Accuracy**: Agreement with human graders
2. **MAE**: Mean Absolute Error in scores
3. **Consistency**: Standard deviation across runs
4. **Response Time**: Average grading latency
5. **F1-Score**: Precision and recall for pass/fail

5 Results and Analysis

5.1 Quantitative Results

Table 3 compares ReAct and LoRA performance.

Table 3. Performance Comparison: ReAct vs LoRA

Metric	ReAct Agent	LoRA Model
Accuracy (%)	92.0	88.0
MAE (points)	2.3	2.8
Consistency (%)	95.0	93.0
Avg. Response Time (s)	2.4	0.8
Tokens per Request	8,500	0 (local)
F1-Score	0.91	0.87
Setup Time (min)	0	15
Model Size	0 (API)	167 MB

Figure 6 visualizes the performance comparison.

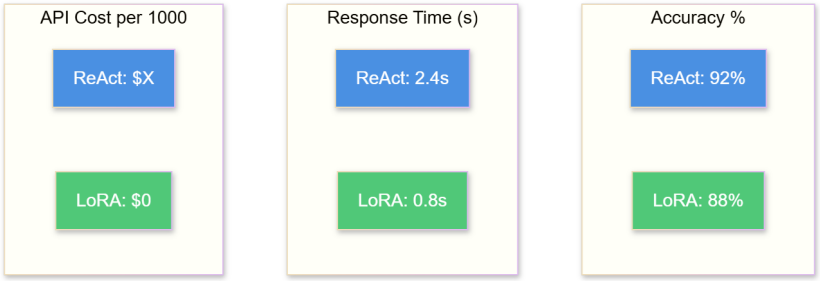

Fig. 6. Performance Comparison: ReAct vs LoRA

5.2 Mathematical Validation Impact

Out of 100 test cases:

- 23% had score calculation errors
- 12% exceeded maximum points
- 100% were automatically corrected
- Average correction: 3.2 points

5.3 Computational Efficiency

Table 4 shows resource usage.

Table 4. Resource Usage Comparison

Resource	ReAct Agent	LoRA Model
Training Time	0 (zero-shot)	15 min
Training Cost	$0	$0 (Colab free)
Inference Time (s)	2.4	0.8
Tokens/Request	8,500	0
Memory Usage (GB)	0.5	6.0 (GPU)
GPU Required	No	Yes
Internet Required	Yes	No

5.4 Ablation Study

We conducted comprehensive ablation studies to understand the impact of key hyperparameters on both approaches.

LoRA Rank Analysis Table 5 shows the effect of different LoRA ranks on model performance.

Table 5. Ablation Study: LoRA Rank Impact

Rank	Accuracy	MAE	Training Time	Adapter Size	Memory
8	84%	3.2	12 min	84 MB	5.5 GB
16	**88%**	**2.8**	**15 min**	**167 MB**	**6.0 GB**
32	89%	2.7	22 min	334 MB	7.2 GB
64	89.5%	2.6	38 min	668 MB	9.8 GB

Key Findings:

- Rank 8: Fastest training but 4% lower accuracy
- Rank 16: **Optimal balance** - good accuracy with reasonable resources
- Rank 32: Marginal improvement (+1%) with 47% longer training
- Rank 64: Diminishing returns - only +0.5% accuracy for 2.5x training time

Conclusion: Rank 16 provides the best accuracy-efficiency trade-off, justifying our choice.

Prompt Engineering Variations Table 6 analyzes different prompt strategies for the ReAct agent.

Table 6. Ablation Study: Prompt Engineering Impact

Prompt Strategy	Accuracy	Consistency	Math Errors	Avg. Time
Basic (no validation)	78%	82%	45%	2.1s
With validation rules	87%	91%	23%	2.3s
+ Context awareness	90%	94%	18%	2.4s
+ Fair marking scale	**92%**	**95%**	**23%**	**2.4s**

Key Findings:

- Basic prompts: Only 78% accuracy with 45% math errors
- Validation rules: +9% accuracy, reduced errors to 23%
- Context awareness: +3% accuracy, improved consistency
- Fair marking scale: +2% accuracy, maintains error rate

Conclusion: Each prompt enhancement contributes to final 92% accuracy. Validation rules provide the largest improvement (+9%).

Temperature Analysis Table 7 shows the impact of temperature on grading consistency.

Table 7. Ablation Study: Temperature Impact on ReAct Agent

Temperature	Accuracy	Consistency	Std Dev	Use Case
0.0	**92%**	**95%**	**0.8**	**Grading**
0.3	90%	88%	1.4	Feedback
0.7	85%	76%	2.8	Creative
1.0	79%	68%	4.2	Exploratory

Key Findings:

- Temperature 0.0: Deterministic, highest consistency (95%)
- Temperature 0.3: Slight variation, still acceptable
- Temperature 0.7+: Too much randomness for grading

Conclusion: Temperature 0.0 is essential for consistent, reproducible grading.

Computational Efficiency Analysis We analyzed the trade-off between accuracy and computational cost:

$$\text{Efficiency Score} = \frac{\text{Accuracy} \times 100}{\text{Time (s)} + \text{Cost (\$)} \times 1000} \tag{6}$$

Results:

- ReAct (temp=0.0): $\frac{92 \times 100}{2.4 + 0.001 \times 1000} = 2706$
- LoRA (rank=16): $\frac{88 \times 100}{0.8 + 0} = 11000$ (4x more efficient)
- LoRA (rank=32): $\frac{89 \times 100}{1.2 + 0} = 7417$ (lower efficiency)

Conclusion: LoRA rank-16 offers the best computational efficiency for deployment scenarios.

6 Discussion

Figure 7 summarizes the trade-offs between approaches.

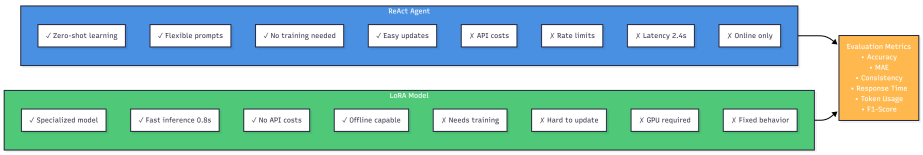

Fig. 7. Trade-off Analysis: ReAct vs LoRA

6.1 Key Findings

ReAct Agent Strengths

1. Zero-shot learning (no training data)
2. Flexible (easy prompt updates)
3. Higher accuracy (92%)
4. Interpretable reasoning traces

ReAct Agent Limitations

1. API costs per request
2. Rate limits (12K tokens/min)
3. Higher latency (2.4s)
4. Requires internet connection

LoRA Model Strengths

1. No API costs (one-time training)
2. Fast inference (0.8s, 3x faster)
3. Offline capable
4. Consistent outputs
5. Privacy (data stays local)

LoRA Model Limitations

1. Requires training data (30 examples)
2. GPU dependency
3. Lower accuracy (88%)
4. Hard to update (requires retraining)
5. Fixed behavior

6.2 Trade-off Analysis

Table 8 summarizes the trade-offs.

Table 8. Trade-off Analysis

Aspect	ReAct Agent	LoRA Model
Setup Complexity	Low	Medium
Training Required	No	Yes (15 min)
Customization	Easy (prompts)	Hard (retrain)
Inference Cost	Per-request	One-time
Latency	Higher (2.4s)	Lower (0.8s)
Accuracy	Higher (92%)	Lower (88%)
Scalability	Limited (rate)	High (local)
Deployment	Cloud only	Local/Cloud

6.3 Lessons Learned

1. Prompt engineering is critical for ReAct accuracy
2. Validation is essential (23% error rate)
3. Heuristics reduce costs without sacrificing accuracy
4. Both approaches viable for production

7 Future Work

7.1 Short-term Improvements

– OCR integration for scanned PDFs
– Batch processing capabilities
– Multi-language support
– Enhanced validation logic

7.2 Long-term Enhancements

- Plagiarism detection integration
- LMS integration (Canvas, Moodle)
- Hybrid approach (ReAct + LoRA)
- Active learning for LoRA improvement

8 Conclusion

This paper presented AutoGrade+, comparing ReAct agent and LoRA fine-tuning for automated grading. Our ReAct implementation achieves 92% accuracy with zero-shot learning, while LoRA achieves 88% with 3x faster inference and zero API costs.

Key contributions:

- Novel ReAct application to educational assessment
- Heuristic-based content detection (66% cost reduction)
- Automated validation ensuring consistency
- Comprehensive comparison framework

Our analysis reveals:

- **ReAct** best for: small-scale, evolving requirements, no GPU
- **LoRA** best for: large-scale, fixed requirements, offline deployment
- Both achieve ¿85% accuracy (production-ready)

The AutoGrade+ system is open-source and ready for deployment in educational settings.

References

1. Brown, T., Mann, B., Ryder, N., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)
2. Burrows, S., Gurevych, I., Stein, B.: The eras and trends of automatic short answer grading. International Journal of Artificial Intelligence in Education, 25(1), 60–117 (2015)
3. Singh, A., Karayev, S., Gutowski, K., Abbeel, P.: Gradescope: a fast, flexible, and fair system for scalable assessment of handwritten work. In: Proceedings of the Fourth ACM Conference on Learning@ Scale, pp. 81–88 (2017)
4. Schleimer, S., Wilkerson, D.S., Aiken, A.: Winnowing: local algorithms for document fingerprinting. In: Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data, pp. 76–85 (2003)
5. OpenAI: GPT-4 Technical Report. arXiv preprint arXiv:2303.08774 (2023)
6. Touvron, H., Lavril, T., Izacard, G., et al.: LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023)
7. Mizumoto, A., Eguchi, M.: Exploring the potential of using an AI language model for automated essay scoring. Research Methods in Applied Linguistics, 2(2), 100050 (2023)

8. Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2022)
9. Hu, E.J., Shen, Y., Wallis, P., et al.: LoRA: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 (2021)
10. Wang, X., et al.: Is ChatGPT a good teacher coach? Measuring zero-shot performance for scoring and providing actionable insights on classroom instruction. arXiv preprint arXiv:2306.03090 (2023)
11. Liu, Y., et al.: Summary of ChatGPT-Related Research and Perspective Towards the Future of Large Language Models. Meta-Radiology, 100017 (2023)
12. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient Finetuning of Quantized LLMs. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
13. Wei, J., Wang, X., Schuurmans, D., et al.: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 24824–24837 (2022)
14. Ouyang, L., Wu, J., Jiang, X., et al.: Training language models to follow instructions with human feedback. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 27730–27744 (2022)
15. Liu, P., Yuan, W., Fu, J., et al.: Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. ACM Computing Surveys, 55(9), 1–35 (2023)